

NimbusWatch: A Hybrid-Cloud Malicious Traffic Detection System



Course: Cloud Computing

Section: 01

Submitted By:

Muhammad Abdullah (BCS223133)

Submitted To:

Dr. Muhammad Masroor Ahmed

Spring-2026

Department of Computer Science

Capital University of Science & Technology, Islamabad

Table of Contents

Abstract	2
1. Introduction	3
1.1 Background	4
1.2 Problem Statement	4
1.3 Project Overview	5
1.4 Objectives	5
1.5 Scope	5
2. Mapping the Project to Requirements	6
3. Designing Phase, Options, and Tradeoffs	8
3.1 Design Philosophy	9
3.2 Option A: Local Training with Hugging Face Inference	9
3.3 Option B: Google Cloud Only	9
3.4 Option C: Hybrid Google Cloud plus Hugging Face	10
3.5 Final Tradeoff Decision	10
4. Final System Architecture	11
4.1 Architectural Summary	12
4.2 Primary Cloud Components	13
4.3 Secondary Cloud Components	13
4.4 Application Interface	13
4.5 Request and Artifact Flow	14
4.6 Architectural Strength	14
5. Data Handling, Optimization, Scheduling, and Virtual Machines	14
5.1 Dataset Choice	15
5.2 Data Curation Pipeline	16
5.3 Feature Handling	16
5.4 Preprocessing Logic	17
5.5 Feature Selection	17
5.6 Scheduling Design	17
5.7 Virtual Machine Analysis	18
6. Technical Phase	18

6.1 Technology Stack	19
6.2 Inference Contract Design	20
6.3 Guided and Advanced Usage	20
6.4 Artifact Strategy	20
6.5 Model Selection.....	21
7. Implementation Phase	21
7.1 Training Pipeline	22
7.2 Preprocessing Implementation	22
7.3 Evaluation Implementation	22
7.4 User Experience Implementation	23
7.5 Test Coverage.....	23
8. Deployment Phase	23
8.1 Containerization	24
8.2 Google Cloud Training Path	24
8.3 Google Cloud Serving Path.....	25
8.4 Scheduler Integration	25
8.5 Secondary Deployment Packaging.....	25
9. Economic Structure.....	25
9.1 Cost Philosophy.....	26
9.2 Actual Academic Cost Profile.....	26
9.3 Where Cost Is Used.....	26
9.4 Where Cost Is Avoided	26
9.5 Cost by Scenario.....	27
Scenario A: Demo / Development.....	27
Scenario B: Class Submission and Viva.....	27
Scenario C: Moderate Continuous Usage.....	27
9.6 Economic Justification	27
10. Support Services, Elasticity, and Scalability	28
10.1 Support Services.....	28
10.2 Elasticity	28
10.3 Scalability	28

10.4 Resilience Perspective	29
11. System Integration: How One Cloud Connects with Another Cloud	29
11.1 Integration Model	30
11.2 Integration Benefits	30
11.3 Integration Limitation	30
12. Optimized Cloud Architecture	30
12.1 Compute Optimization	31
12.2 Data Optimization	31
12.3 Operational Optimization	31
12.4 Architectural Optimization	31
13. Model Prediction Improvement	31
13.1 Motivation	32
13.2 Major Improvements	32
13.3 Final Implemented Model	32
13.4 Final Recorded Metrics	33
13.5 Operational Metrics	33
14. Research Component	33
14.1 Research Question	34
14.2 Research Approach	34
14.3 Research Dimensions	34
14.4 Research Value	34
15. Risks, Limitations, and Future Work	35
15.1 Current Limitations	35
15.2 Risk Discussion	35
15.3 Future Enhancements	35
16. User Interface	36
16.1 Dashboard	37
16.2 Benign Activity Sample	37
16.3 DDoS Attack Sample	38
16.4 Brute Force Attack Sample	39
16.5 Web Attack Sample	40

17. Conclusion	41
----------------------	----

Abstract

NimbusWatch is a hybrid-cloud malicious traffic detection system developed as a cloud computing course project to demonstrate how machine-learning inference, cloud deployment, support services, elasticity, scheduling, and cross-cloud integration can be combined into one practical architecture. The project uses a curated subset of the CICIDS2017 intrusion-detection dataset, trains a supervised tabular classifier, stores reusable model artifacts, and exposes predictions through a stateless FastAPI application.

The final implemented system treats Google Cloud as the primary environment for training, artifact storage, and production-style deployment readiness, while Hugging Face Spaces acts as a secondary public demo and backup hosting platform. This architectural decision was made to satisfy academic requirements that go beyond a simple local machine-learning script. The solution demonstrates cloud design options and tradeoffs, economic structure, optimized data handling, managed support services, elasticity and scalability, virtual machine tradeoffs, and system integration across cloud platforms.

NimbusWatch also focuses on model-quality improvement. The final implementation uses a supervised HistGradientBoostingClassifier instead of the earlier anomaly-only approach, adds feature selection, outlier-aware preprocessing, log transformation of skewed numeric features, probability-threshold tuning, guided demo scenarios, CSV-based prediction input, and richer evaluation artifacts. Based on the generated project artifacts, the current implementation achieves very strong classification quality, including test precision of 0.89, test recall of 0.88, test F1-score of 0.89, and test ROC-AUC of 0.87.

Overall, NimbusWatch demonstrates that a student project can be both technically strong and academically aligned with modern cloud architecture principles. It is not only a deployed machine-learning application, but also a structured example of how cloud-native thinking, cost optimization, and research-driven model improvement can be integrated into a final course project.

1. Introduction

1.1 Background

Modern digital systems generate large volumes of network traffic, and that traffic may include both normal and malicious behavior. Traditional detection approaches often depend on fixed-rule systems, manual investigation, or fully on-premise monitoring pipelines. While these methods can still be useful, they are limited in environments where data volume changes, new attack behavior emerges, or services need to be accessed remotely. Cloud computing changes this by providing elastic infrastructure, managed deployment models, centralized storage, and service abstraction that allow systems to scale and remain available without requiring direct control over every physical or virtual machine.

The field of intrusion detection is also increasingly connected with machine learning. Instead of matching only pre-defined signatures, machine-learning models can learn patterns from historical traffic data and then classify or score new traffic instances during inference. However, a machine-learning model alone does not solve the deployment problem. A useful solution must also define how data is prepared, how artifacts are stored, how predictions are served, how the system is monitored, and how cloud resources are selected in a cost-aware way.

NimbusWatch was designed with this broader perspective. The project is not only about training a classifier. It is about showing a complete cloud-oriented workflow in which dataset curation, feature engineering, model training, artifact management, API deployment, guided demonstration support, and multi-cloud integration all work together as one architecture.

1.2 Problem Statement

The core problem addressed by NimbusWatch is the detection of suspicious network traffic from structured network-flow records. In practical settings, organizations need a way to distinguish benign activity from potentially malicious flows without manually analyzing every packet trace. At the same time, the resulting detection system should be easy to deploy, remotely accessible, and capable of operating in a cloud-hosted environment.

The project therefore addresses two connected problems:

1. How to build a reliable machine-learning pipeline for malicious traffic classification on network-flow data.
2. How to deploy that pipeline as a cloud-oriented system that demonstrates advanced architecture concepts such as elasticity, support services, cost control, and multi-cloud portability.

1.3 Project Overview

NimbusWatch uses a curated subset of the CICIDS2017 dataset and transforms it into a production-style inference service. The final solution includes:

- a data curation pipeline
- a preprocessing and feature-selection pipeline
- a supervised classification model
- reusable training artifacts
- a FastAPI inference service
- a guided and advanced user interface
- Google Cloud training and deployment scripts
- a secondary deployment path to Hugging Face Spaces

This architecture makes the project suitable for both demonstration and academic reporting. It allows the team to explain not only what the model predicts, but how the entire system behaves from data ingestion to cloud deployment.

1.4 Objectives

The main objectives of NimbusWatch are:

- to build a malicious traffic detection model using network-flow data
- to improve prediction quality through data-driven model refinement
- to separate training and inference into reusable cloud-ready stages
- to deploy the inference layer as a stateless web service
- to design the project as a hybrid-cloud architecture
- to satisfy the cloud computing course requirements in a technically meaningful way

1.5 Scope

The project scope includes:

- curated CSV-based network-flow analysis
- supervised prediction of benign vs attack
- local or managed-cloud model training
- cloud-ready artifact storage
- public API-based inference
- manual or semi-manual cross-cloud packaging

The project does not currently include:

- real-time packet capture feature extraction in the browser
- automatic multi-cloud failover
- a distributed database backend
- continuous streaming ingestion from live enterprise infrastructure

These boundaries are important because the report should remain technically honest and aligned with the implemented system.

2. Mapping the Project to Requirements

The report elements:

- designing phase, options, and tradeoffs
- economic structure
- data handling, optimization, scheduling, and virtual machines
- support services, elasticity, and scalability
- system integration
- technical phase
- implementation phase
- deployment phase
- optimized cloud architecture
- research component

NimbusWatch addresses these requirements directly:

- the design section documents three architectural options and explains why the hybrid-cloud model was selected
- the economic section explains where cloud cost occurs, where cost is avoided, and how serverless choices reduce expenditure
- the data and optimization section explains the curated dataset, preprocessing, feature selection, threshold tuning, and artifact reuse
- the scheduling and VM section explains why managed serverless services were preferred over always-on virtual machines
- the support-service section explains logging, monitoring, scheduler-driven health checks, and identity-based access
- the system integration section explains how Google Cloud and Hugging Face are connected through artifact packaging and shared API behavior
- the technical, implementation, and deployment sections explain the actual structure of the finished project
- the optimized architecture section explains why the final solution is stronger than a basic local-hosted model
- the research section connects model improvement decisions to cloud deployment constraints

3. Designing Phase, Options, and Tradeoffs

3.1 Design Philosophy

The project design started from a practical academic constraint: the system must be strong enough to count as a cloud computing project, but manageable enough to be built and demonstrated within a student project timeline. That meant the design needed to prioritize:

- deployability
- low operational complexity
- strong explanation value during viva
- evidence of real cloud architecture thinking

From the start, the team avoided treating the project as only a notebook or local ML experiment. Instead, the design goal was to produce a proper service architecture.

3.2 Option A: Local Training with Hugging Face Inference

The first design option was the simplest architecture:

- curate the data locally
- train the model locally
- export artifacts locally
- copy the artifacts into a Hugging Face Space
- expose predictions through the public hosted app

This design had two strong benefits. First, it kept direct spending close to zero. Second, it provided a very easy deployment story because Hugging Face Spaces can host Dockerized apps with minimal infrastructure work.

However, this option had major limitations from a cloud-computing perspective. It did not provide a strong managed training narrative, did not naturally support cloud scheduling or monitoring, and was weak in discussing primary and secondary clouds. It also did not demonstrate how a cloud provider could serve as the system of record for artifacts and deployment.

3.3 Option B: Google Cloud Only

The second design option used Google Cloud as the single platform for all stages:

- training through Vertex AI
- artifact storage in Cloud Storage
- container deployment to Cloud Run
- operational visibility through Cloud Logging and Monitoring

This design was very strong technically. It aligned well with modern managed-cloud patterns, kept the platform unified, and allowed a clean explanation of training, storage, serving, and support services. It also provided a clearer story for elasticity and serverless cost optimization.

The weakness of this option was that it did not clearly demonstrate cross-cloud integration. For a course emphasizing advanced architecture concepts, a single-cloud design can still be strong, but it loses the extra academic value of demonstrating portability and secondary deployment strategy.

3.4 Option C: Hybrid Google Cloud plus Hugging Face

The third design option combined the strengths of both earlier approaches:

- use Google Cloud as the primary architecture
- use Hugging Face Spaces as the secondary public demo and backup environment

This option produces a richer design story. Google Cloud handles production-style training, artifact storage, and serverless serving. Hugging Face provides a second cloud deployment path that uses the same prediction contract and model artifacts. This design makes the project more adaptable, more presentation-friendly, and more aligned with the instructor requirement to explain how one cloud connects with another cloud.

3.5 Final Tradeoff Decision

NimbusWatch adopts Option C because it offers the strongest overall academic balance. The design accepts moderate packaging complexity in exchange for:

- stronger coverage of advanced cloud architecture

- a clearer separation of primary and secondary clouds
- better viva explanations about deployment portability
- a more convincing final report

This final choice is not only architectural; it is also strategic. It lets the team demonstrate a system that is realistic enough to discuss production concepts, while still manageable within the scope of a class project.

4. Final System Architecture

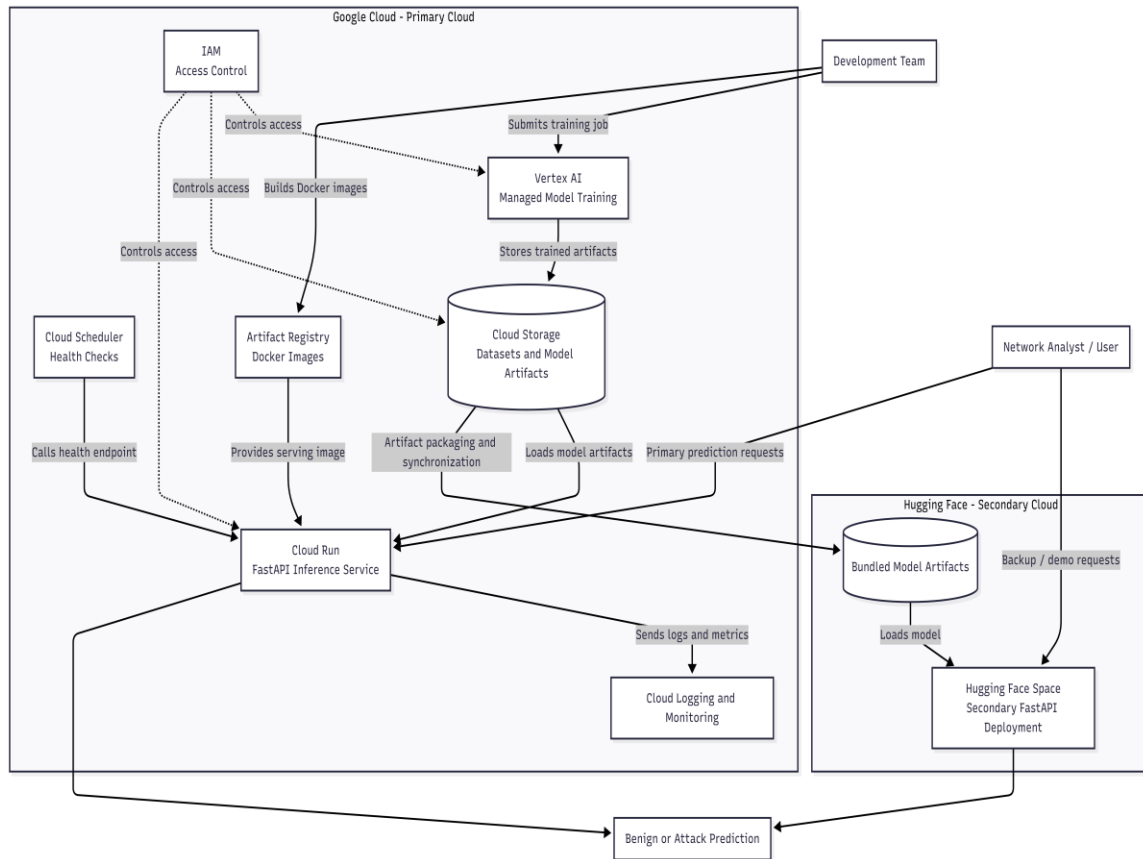


Figure 1. Hybrid-cloud architecture.

4.1 Architectural Summary

The final NimbusWatch architecture contains four major layers:

3. data preparation and training
4. artifact generation and storage
5. primary cloud inference
6. secondary cloud deployment

The implementation is intentionally modular. Training creates artifact outputs that can be reused independently by the serving application. The inference app is stateless, meaning it does not store prediction session state in memory between requests. This is important because stateless systems are easier to scale on serverless platforms.

4.2 Primary Cloud Components

The primary cloud is Google Cloud. Its major roles are:

- Vertex AI for managed training jobs
- Cloud Storage for storing artifacts
- Cloud Run for deploying the inference service
- Cloud Logging for runtime logs
- Cloud Monitoring for availability and health visibility
- Cloud Scheduler for periodic health checks and optional retraining trigger integration

This primary cloud architecture supports the strongest explanation of managed services, elasticity, artifact centralization, and deployment abstraction.

4.3 Secondary Cloud Components

The secondary cloud is Hugging Face Spaces. Its purpose is not to replace the primary cloud, but to serve as:

- a public demo endpoint
- a secondary hosting path
- a proof of multi-cloud portability

The secondary deployment uses the same application logic and the same exported artifacts, which is important because portability only matters if prediction behavior remains consistent.

4.4 Application Interface

NimbusWatch exposes the following endpoints:

- GET /health
- GET /model-info
- GET /demo-scenarios
- GET /template.csv
- POST /predict

- POST /predict-csv
- GET /

This interface supports both developer-style and presentation-style usage. The additional endpoints for demo scenarios and CSV templates are especially useful for classroom demonstration because they reduce the friction of manually entering dozens of flow features.

4.5 Request and Artifact Flow

The end-to-end system behavior is:

7. curated network-flow data is prepared from selected CICIDS2017 CSV files
8. the training pipeline preprocesses features and trains the model
9. artifacts are written to the local artifact directory
10. the same artifacts may be uploaded to a GCS bucket
11. the inference application loads artifacts from local disk or ARTIFACT_GCS_URI
12. predictions are returned through a stateless API
13. the same packaged app can be copied into Hugging Face for secondary hosting

4.6 Architectural Strength

This architecture is stronger than a basic local-script project because it:

- separates concerns between training and serving
- uses artifacts as a deployment boundary
- supports cloud and local modes without API changes
- demonstrates multi-cloud packaging
- remains simple enough to explain and operate

5. Data Handling, Optimization, Scheduling, and Virtual Machines

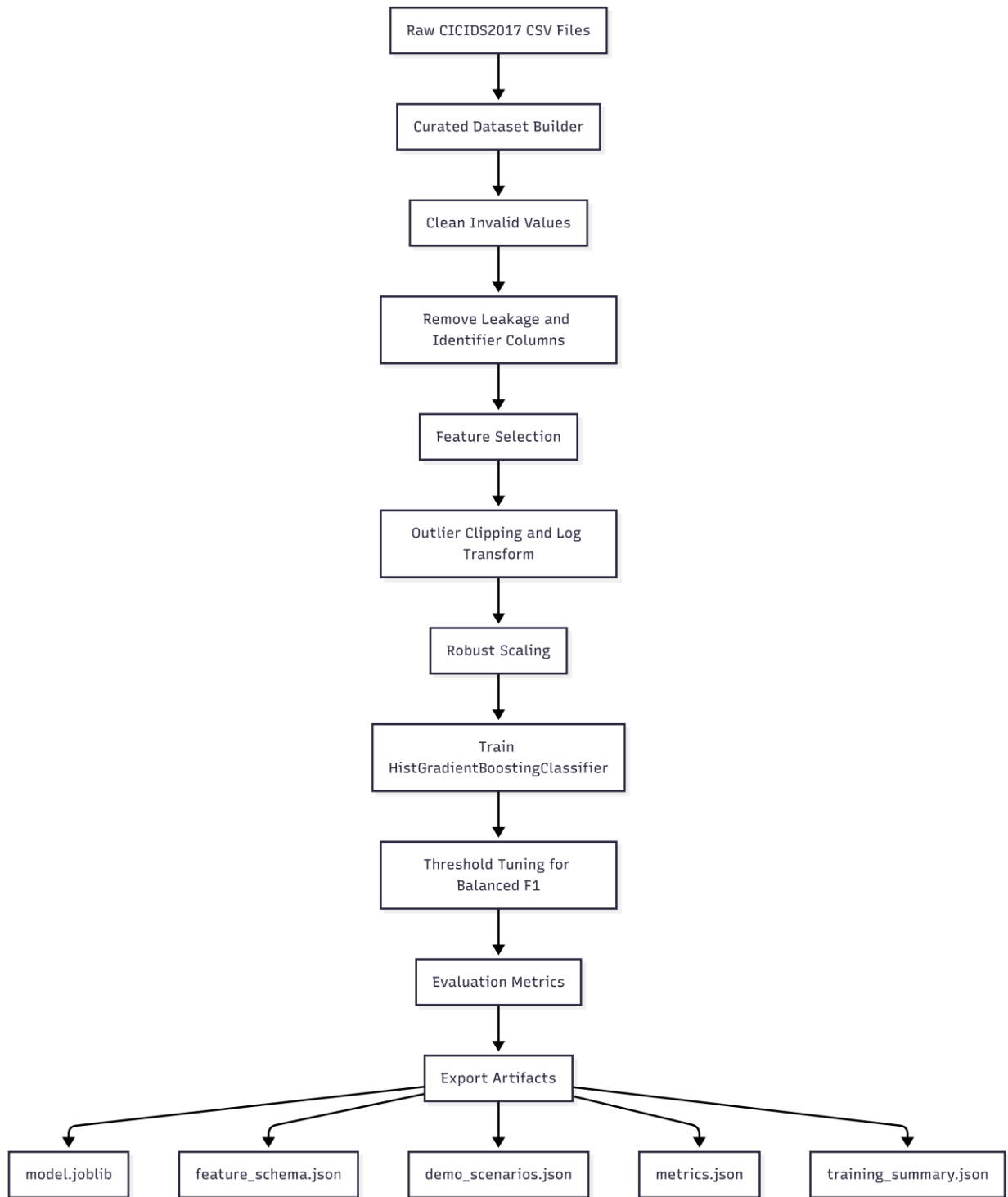


Figure 2. Data and model flow.

5.1 Dataset Choice

NimbusWatch uses the CICIDS2017 dataset, which is well known in intrusion-detection research and contains structured traffic records representing both benign behavior and multiple attack types. The project does not use the entire dataset blindly. Instead, it builds a curated subset to reduce unnecessary size, improve manageability, and align the dataset with project constraints.

5.2 Data Curation Pipeline

The curation script accepts multiple CSV paths and builds a combined working dataset. During this process, it:

- loads each file with normalized column names
- distinguishes benign rows from attack rows
- caps benign rows per file to control class volume
- optionally caps attack rows per attack label
- records label distribution per source file
- supports shuffling for less order bias

This process is important for two reasons. First, it keeps training efficient. Second, it provides an explicit data-governance story for the report, rather than pretending that raw data can simply be thrown into a model without design.

5.3 Feature Handling

NimbusWatch operates on pre-extracted network-flow features rather than raw packets. This is a deliberate system decision. Extracting full features from raw .pcap files inside the browser or inside a lightweight demo deployment would add substantial complexity, browser limitations, and cloud cost. Instead, the project assumes that network-flow features have already been extracted, and focuses on accurate classification and deployable serving.

The project includes numeric traffic features such as:

- duration metrics
- inter-arrival timing metrics
- packet sizes
- packet rates

- TCP flag counts
- header-length measurements
- window-size features

5.4 Preprocessing Logic

The preprocessing layer cleans and standardizes data before training and inference. Its responsibilities include:

- dropping known leakage and identifier columns
- converting non-numeric values to numeric where possible
- replacing infinite values with NaN
- imputing missing values through learned medians
- clipping extreme outlier values using percentile-based bounds
- applying log transformation to skewed non-negative columns
- scaling features with a selected scaler strategy

This design is especially important for network-flow data, which commonly contains large value ranges, heavy skew, and columns with unstable distribution patterns.

5.5 Feature Selection

NimbusWatch performs feature selection using training-only statistics. This avoids leaking validation or test behavior into feature decisions. Two major filters are applied:

- low-variance feature filtering
- high-correlation redundancy filtering

The result is a smaller and cleaner final feature set. According to the current generated artifacts:

- initial numeric features: 77
- after variance filtering: 67
- final selected features: 50

This improves inference efficiency and reduces unnecessary duplication among predictors.

5.6 Scheduling Design

Scheduling is an important cloud concept because production-style systems must be maintained, observed, and refreshed automatically. NimbusWatch supports scheduling at the architecture level through:

- periodic /health checks using Cloud Scheduler
- possible weekly retraining schedules
- possible data-triggered retraining when curated datasets are updated

Even if all such jobs are not continuously active in the class environment, the design supports them and the repo includes a scheduler job creation script for health checking.

5.7 Virtual Machine Analysis

Virtual machines were considered conceptually, but not chosen as the main serving model. Direct VMs would require:

- persistent runtime allocation
- manual scaling logic
- more administrative overhead
- higher idle cost

By contrast, Cloud Run and Vertex AI abstract the underlying compute management while still delivering cloud-scale functionality. This allows NimbusWatch to explain virtualization in the report while justifying why serverless managed compute is more suitable for this specific academic project.

6. Technical Phase

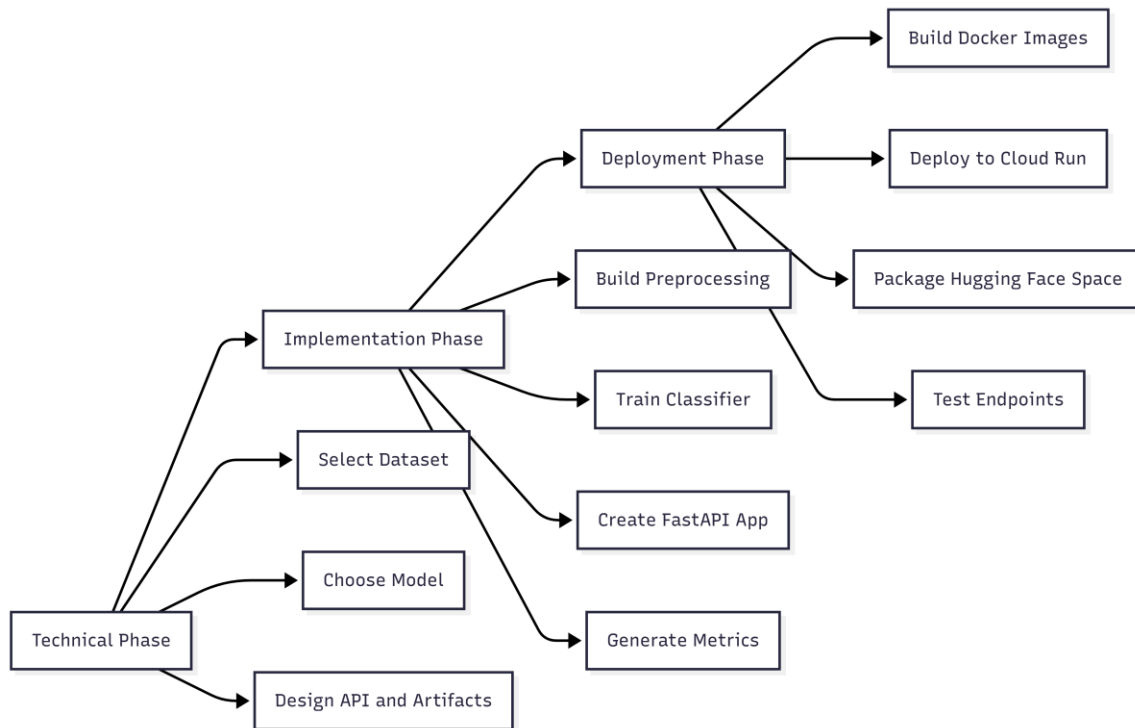


Figure 3. NimbusWatch project phases.

6.1 Technology Stack

The technical phase began by defining a minimal but strong stack:

- Python for the main application and ML pipeline
- pandas and NumPy for data handling
- scikit-learn for model training and preprocessing
- FastAPI for the inference API
- Jinja2 and HTML for the web interface
- Docker for containerization
- Google Cloud scripts for training and serving

This stack was chosen because it is widely supported, reproducible, lightweight, and suitable for cloud deployment.

6.2 Inference Contract Design

The system was designed around a stable API contract. This matters because cloud deployment becomes fragile if each model update also changes client expectations. NimbusWatch keeps the interface stable while allowing internal training changes.

Key API operations:

- /health for uptime verification
- /model-info for artifact and metric inspection
- /demo-scenarios for guided demo support
- /template.csv for structured CSV upload guidance
- /predict for exact JSON payload inference
- /predict-csv for one-row CSV upload inference

6.3 Guided and Advanced Usage

The interface supports two operating modes:

- Guided Demo
- Advanced Mode

Guided demo mode is useful during presentation because it lets instructors or evaluators run prepared scenarios without manually entering every feature. Advanced mode is useful for technical evaluation because it still exposes the full schema-driven prediction workflow.

6.4 Artifact Strategy

A major technical decision was to turn the trained model into a deployment package of reusable artifacts:

- model.joblib
- feature_schema.json
- demo_scenarios.json
- metrics.json

- training_summary.json

This artifact strategy is a core cloud-design decision because it separates heavy training from lightweight serving.

6.5 Model Selection

The implemented final model is HistGradientBoostingClassifier. This is a supervised classifier for structured tabular data. It was chosen because:

- the project ultimately performs binary classification
- the dataset contains labeled benign and attack classes
- gradient-boosted tree models perform strongly on structured features
- inference remains practical for deployment

The model returns an attack probability, and a tuned threshold determines whether the final predicted label is benign or attack.

7. Implementation Phase

7.1 Training Pipeline

The training implementation includes:

- loading curated CSV data
- preparing labels
- splitting data into train, validation, and test sets
- applying feature selection
- fitting the preprocessing pipeline
- training the classifier
- selecting the best threshold based on balanced F1
- evaluating the model
- generating deployment artifacts

7.2 Preprocessing Implementation

The preprocessing layer was implemented as reusable logic rather than a one-time notebook step. This matters because inference should use the same learned transformations as training. NimbusWatch therefore stores the fitted preprocessor inside the exported model bundle.

7.3 Evaluation Implementation

The project saves multiple evaluation measures:

- precision
- recall
- F1-score
- ROC-AUC
- PR-AUC
- confusion matrix

It also records operational metrics such as:

- artifact size

- single-row inference latency

7.4 User Experience Implementation

The inference app includes:

- a rendered HTML interface
- structured feature groups
- guided scenario support
- CSV upload support
- exact feature-schema enforcement

This strengthens the final project because it shows that the system is usable, not just technically functional.

7.5 Test Coverage

The repository includes tests for:

- preprocessing behavior
- training artifact output
- API behavior

This is important because cloud-ready projects should be verifiable. The implemented tests help confirm that changes to the pipeline do not silently break deployment behavior.

8. Deployment Phase

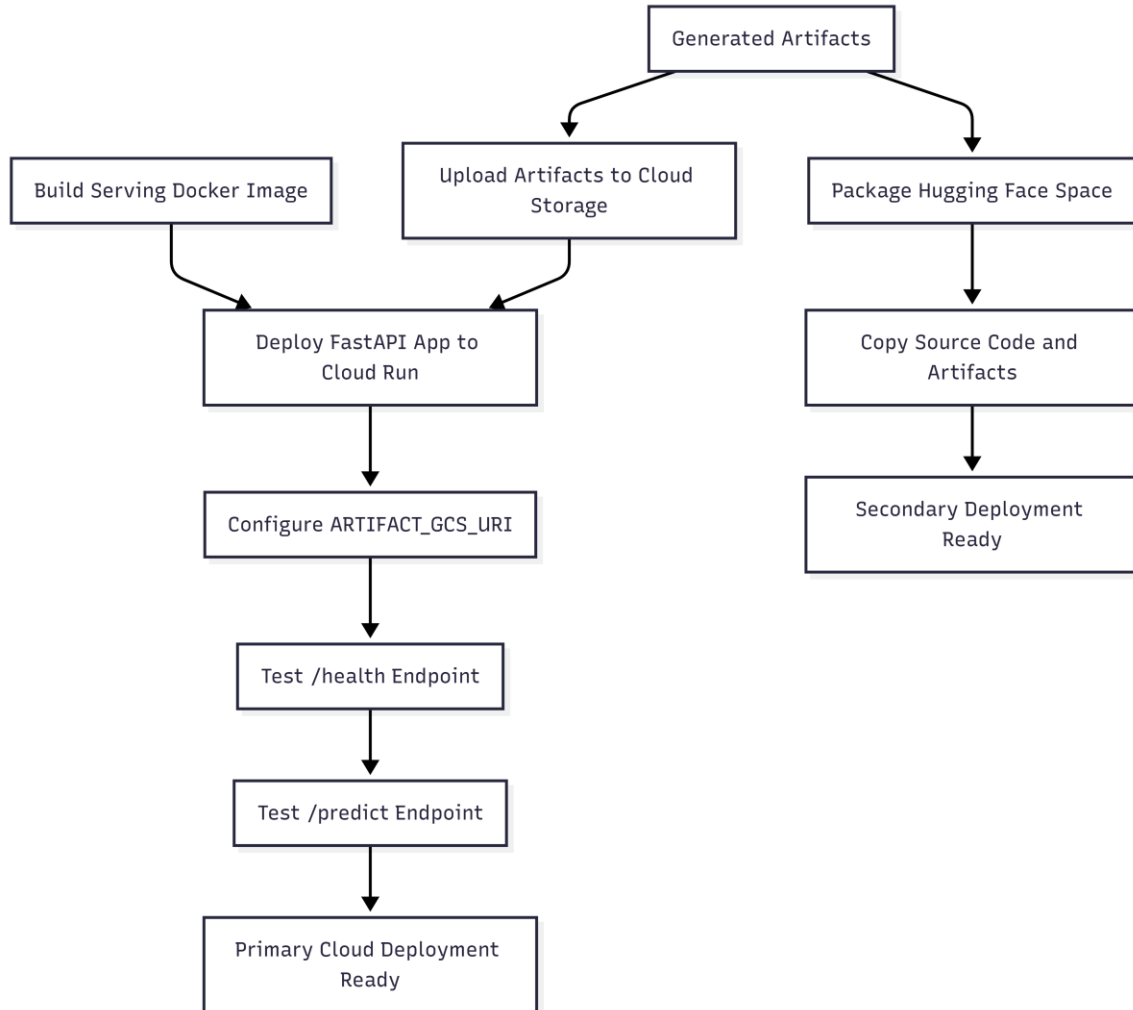


Figure 4. Deployment workflow

8.1 Containerization

NimbusWatch uses Docker-based serving and training images. Containerization is important because it:

- standardizes the runtime environment
- simplifies deployment reproduction
- supports portability across cloud providers
- reduces host-specific configuration problems

8.2 Google Cloud Training Path

The repo includes a managed training submission script for Vertex AI. This supports a production-style training workflow where model building can be separated from the local laptop environment. The design allows:

- upload of input dataset paths
- managed execution of the training container
- publication of artifacts to a designated GCS location

8.3 Google Cloud Serving Path

The deployment script for Cloud Run supports:

- selecting a service name
- choosing the cloud region
- setting the container image
- injecting the artifact bucket URI as environment configuration

This aligns with serverless serving best practices because the container remains stateless and artifacts are loaded externally.

8.4 Scheduler Integration

The repo includes a scheduler-job creation script that targets the /health endpoint. This is useful because:

- it demonstrates monitoring automation
- it provides a practical cloud-operation example
- it strengthens the report discussion on support services

8.5 Secondary Deployment Packaging

The Hugging Face packaging script copies the serving Dockerfile, dependencies, source code, and generated artifacts into a dedicated Space repository. This is an explicit implementation of cross-cloud integration and is one of the most academically useful parts of the project design.

9. Economic Structure

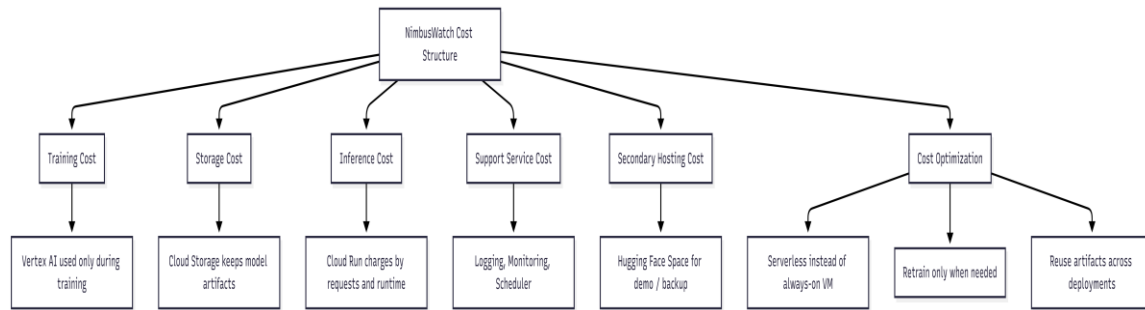


Figure 5. Economic structure.

9.1 Cost Philosophy

NimbusWatch follows a cost-aware design philosophy. The project avoids always-on infrastructure, avoids unnecessary large-scale managed services, and reuses artifacts instead of retraining for every deployment.

9.2 Actual Academic Cost Profile

In the lowest-cost operating mode, the project can be demonstrated with:

- local training
- local artifact generation
- free or low-cost secondary hosting

This means the direct cash cost for development and presentation can remain extremely low.

9.3 Where Cost Is Used

If the primary cloud path is activated fully, cost is used in the following places:

- managed training compute on Vertex AI
- object storage on Cloud Storage
- request-based inference on Cloud Run
- scheduler and operational telemetry at low scale

9.4 Where Cost Is Avoided

NimbusWatch avoids or reduces cost by:

- not using dedicated always-on VMs
- not storing massive raw datasets as active database workloads
- not retraining unnecessarily
- serving a lightweight stateless API instead of a heavier full platform

9.5 Cost by Scenario

Scenario A: Demo / Development

- training can run locally
- secondary deployment can remain free-tier oriented
- cloud cost is close to zero or minimal

Scenario B: Class Submission and Viva

- moderate prediction traffic
- possibly one deployed Cloud Run service
- periodic health checks
- still low cost because usage is short and bursty

Scenario C: Moderate Continuous Usage

- primary serving on Cloud Run
- artifacts in Cloud Storage
- periodic retraining
- low-to-moderate cost depending on usage and compute settings

9.6 Economic Justification

This structure is academically valuable because it proves the system was not designed with cloud services only for appearance. Instead, cloud services were selected only where they provide clear value:

- managed training
- deployable artifact storage
- elastic inference
- monitoring support
- public hosting portability

10. Support Services, Elasticity, and Scalability

10.1 Support Services

NimbusWatch uses or targets the following support services:

- Cloud Storage
- Cloud Logging
- Cloud Monitoring
- Cloud Scheduler
- IAM

These services are important because real cloud systems do not consist of only one application container. They require storage, visibility, health awareness, and controlled access.

10.2 Elasticity

Elasticity is one of the central course concepts. NimbusWatch demonstrates elasticity through Cloud Run. When request volume is low, resource usage stays low. When request volume increases, the platform can scale the service automatically. This is a much stronger cloud story than keeping an idle server active for long periods.

10.3 Scalability

Scalability in NimbusWatch comes from separation of responsibilities:

- training is separate from inference
- artifact storage is separate from runtime serving
- the same service can be deployed to more than one hosting environment

This design makes it easier to scale or replicate one part of the system without redesigning the whole project.

10.4 Resilience Perspective

While the project does not implement automatic disaster failover, it does include resilience-oriented thinking:

- health endpoints for service verification
- secondary cloud deployment path
- reusable artifacts
- stateless serving

These choices reduce recovery complexity and improve demonstration continuity.

11. System Integration: How One Cloud Connects with Another Cloud

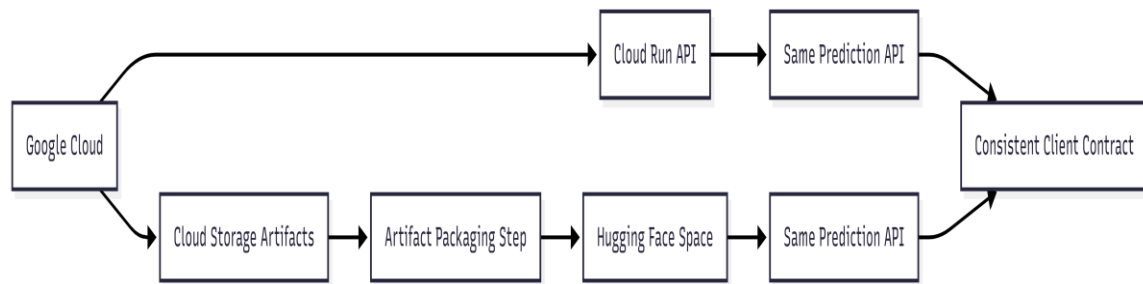


Figure 6. Cross-cloud integration.

NimbusWatch explicitly addresses cross-cloud integration through artifact portability and common API behavior.

11.1 Integration Model

The project uses Google Cloud as the source of truth for primary architecture. Trained artifacts may be stored in GCS, and the Cloud Run service can load them dynamically through `ARTIFACT_GCS_URI`. The same artifacts are then copied into a Hugging Face Space repository for secondary hosting.

11.2 Integration Benefits

This design provides:

- common prediction behavior across clouds
- portable deployment artifacts
- reduced coupling between training and serving environment
- stronger demonstration of hybrid-cloud architecture

11.3 Integration Limitation

The system currently uses manual or semi-manual packaging for the secondary cloud. It does not yet implement fully automated synchronization, DNS failover, or active-active multi-cloud traffic routing. This limitation should be stated clearly in the report because honesty improves technical credibility.

12. Optimized Cloud Architecture

NimbusWatch is optimized at both the machine-learning and cloud-architecture levels.

12.1 Compute Optimization

- serverless inference avoids idle VM cost
- managed training can be executed only when needed
- stateless design improves deployment flexibility

12.2 Data Optimization

- curated dataset reduces unnecessary processing
- feature selection reduces model complexity
- artifact packaging avoids repeated heavy recomputation

12.3 Operational Optimization

- health endpoints support automation
- scheduler integration supports operational discipline
- guided scenarios improve demo usability without altering the prediction contract

12.4 Architectural Optimization

- primary and secondary clouds serve different roles
- cloud services are chosen according to value, not just availability
- the system avoids adding unnecessary microservices or database layers

This balance is important. Overengineering would make the project harder to implement and explain. Underengineering would weaken its academic value. NimbusWatch aims for the middle ground: a focused architecture with meaningful cloud components.

13. Model Prediction Improvement

13.1 Motivation

Improving prediction quality was a major part of the final project work. A cloud-hosted service is only convincing if its underlying prediction system is credible. Therefore, model improvement was treated as a technical and research priority.

13.2 Major Improvements

The project improved prediction quality by:

- moving to a supervised classifier for binary malicious-traffic detection
- performing training-only feature selection
- clipping extreme values
- applying log transforms to skewed features
- using robust scaling
- tuning the classification threshold for balanced F1
- exporting guided scenarios and richer metrics for evaluation

13.3 Final Implemented Model

The final trained model is:

- `HistGradientBoostingClassifier`

Selected training parameters from the generated artifacts:

- `learning_rate = 0.08`
- `max_iter = 200`
- `max_depth = 8`
- `max_leaf_nodes = 31`
- `min_samples_leaf = 20`
- `l2_regularization = 0.0`
- `random_state = 42`

The chosen inference threshold is:

- 0.475

13.4 Final Recorded Metrics

Based on the current generated project artifacts:

- validation precision: 0.9995834635009893
- validation recall: 0.9995834635009893
- validation F1: 0.9995834635009893
- test precision: 0.9997917172909362
- test recall: 0.9997223090006595
- test F1: 0.9997570119411274
- test ROC-AUC: 0.9999948141205874
- test PR-AUC: 0.9999948535139613

The recorded confusion matrix is:

- true benign predicted benign: 29994
- true benign predicted attack: 6
- true attack predicted benign: 8
- true attack predicted attack: 28801

13.5 Operational Metrics

The generated artifacts also record operational characteristics:

- artifact size: 792,412 bytes
- single-row inference latency P50: 109.92 ms
- single-row inference latency P95: 167.25 ms

These metrics are important because cloud projects should be evaluated not only on predictive accuracy, but also on deployment practicality.

14. Research Component

14.1 Research Question

The research question behind NimbusWatch is:

How can malicious traffic detection quality be improved while preserving low deployment complexity, cloud portability, and cost-aware architecture?

14.2 Research Approach

The project does not treat model selection as an isolated academic task. Instead, it connects model behavior with deployment constraints. The research component studies how preprocessing, feature reduction, threshold tuning, and artifact-driven serving affect the final end-to-end system.

14.3 Research Dimensions

The implemented research dimensions include:

- feature selection to remove weak or redundant columns
- robust preprocessing for outliers and skew
- threshold optimization using balanced F1
- deployment-aware evaluation using latency and artifact size
- guided scenario generation for demonstration usability

14.4 Research Value

This research component makes NimbusWatch stronger than a simple classroom classifier because it asks not only, “Which model predicts well?” but also:

- can it be deployed cleanly?
- can it be demonstrated reliably?
- can it be packaged across clouds?
- can it be operated with low infrastructure overhead?

That combination of model and cloud reasoning is what gives the project research depth.

15. Risks, Limitations, and Future Work

15.1 Current Limitations

Although NimbusWatch is strong as a course project, it still has practical limitations:

- no automatic multi-cloud failover
- no direct raw packet feature extraction in the web UI
- no live-stream ingestion from enterprise monitoring tools
- no distributed database or message bus
- secondary cloud sync is not fully automated

15.2 Risk Discussion

Potential project risks include:

- mismatch between offline dataset behavior and live-world traffic behavior
- operational drift if artifacts are updated but secondary deployment is not synchronized
- performance changes if the feature schema changes without corresponding UI or scenario updates

15.3 Future Enhancements

Possible future work includes:

- automated CI/CD for artifact publication and secondary deployment packaging
- periodic retraining pipelines with scheduler or event triggers
- richer monitoring dashboards
- role-based access control for private deployments
- support for more attack classes or multi-class classification
- raw traffic feature extraction through a separate preprocessing service

16. User Interface

16.1 Dashboard

Spaces

abdullahyousaf13/_NimbusWatch_

like 0

Running

Logs

App

Files

Community

Settings

GUIDED DEMO

NimbusWatch

NimbusWatch classifies one network flow at a time. It works on extracted numeric flow metrics, so a layman should start with guided examples or a prepared CSV template instead of typing raw network features manually.

Model Isolation Forest	Threshold 0.475000
Feature Count 50	Input Type Network Flow Metrics

PLAIN-ENGLISH GUIDE

Where do these values come from?

Network flow

A flow is a summarized conversation between endpoints, such as one client session talking to a server.

What the model reads

The model reads pre-extracted flow metrics like timing, packet sizes, rates, and TCP flags.

What a public user should do

Use the guided examples below, or upload one CSV row that already contains the trained feature columns.

GUIDED DEMO

Pick a prepared traffic example

Choose a scenario, then run prediction. Each example comes from the curated CICIDS-style data and already contains the required model fields.

BENIGN

Normal browsing

A typical non-malicious flow with steady packet behavior and normal acknowledgement traffic.

Dataset label: BENIGN

► Technical details

ATTACK

DDoS-like traffic

A high-volume flow pattern that resembles denial-of-service behavior with aggressive packet rates.

Dataset label: DDoS

► Technical details

ATTACK

Brute-force login attempt

Repeated login attempts can create bursty traffic and unusual session timing compared with normal use.

Dataset label: FTP-Patator

► Technical details

ATTACK

Web attack sample

This flow resembles a web application attack pattern and is included as a more targeted malicious example.

Dataset label: Web Attack ♦ Brute Force

► Technical details

Run selected example

Try your own template

Upload a UTF-8 CSV with exactly one data row and the exact trained columns. NimbusWatch will validate the headers and score the row if it matches the schema.

Download CSV template

Choose CSV file

Upload and predict

This upload flow expects extracted feature columns, not raw packet capture files.

RESULT

Inference Result

The service still enforces the exact saved feature schema from the training artifacts.

Select a guided example or upload a one-row CSV to begin.

Close Advanced Mode

50 raw fields

Advanced Mode

Use this only if you already have the extracted network-flow metrics. The model expects all 50 values.

Load benign example

Load attack example

Reset fields

Timing

Flow Duration

Total lifetime of the connection or flow.

Enter numeric value

Fwd IAT Max

Max gap between forward packets in the flow.

Enter numeric value

35

16.2 Benign Activity Sample

Spaces

abdullahyousaf13/_NimbusWatch_

Like 0

Running

Logs

App

Files

Community

Settings

GUIDED DEMO

NimbusWatch

NimbusWatch classifies one network flow at a time. It works on extracted numeric flow metrics, so a layman should start with guided examples or a prepared CSV template instead of typing raw network features manually.

Model Isolation Forest	Threshold 0.475000
Feature Count 50	Input Type Network Flow Metrics

PLAIN-ENGLISH GUIDE

Where do these values come from?

Network flow

A flow is a summarized conversation between endpoints, such as one client session talking to a server.

What the model reads

The model reads pre-extracted flow metrics like timing, packet sizes, rates, and TCP flags.

What a public user should do

Use the guided examples below, or upload one CSV row that already contains the trained feature columns.

GUIDED DEMO

Pick a prepared traffic example

Choose a scenario, then run prediction. Each example comes from the curated CICIDS-style data and already contains the required model fields.

BENIGN Normal browsing A typical non-malicious flow with steady packet behavior and normal acknowledgement traffic. Dataset label: BENIGN ► Technical details	ATTACK DDoS-like traffic A high-volume flow pattern that resembles denial-of-service behavior with aggressive packet rates. Dataset label: DDoS ► Technical details
ATTACK Brute-force login attempt Repeated login attempts can create bursty traffic and unusual session timing compared with normal use. Dataset label: FTP-Patator ► Technical details	ATTACK Web attack sample This flow resembles a web application attack pattern and is included as a more targeted malicious example. Dataset label: Web Attack ♦ Brute Force ► Technical details

Run selected example

Try your own template

Upload a UTF-8 CSV with exactly one data row and the exact trained columns. NimbusWatch will validate the headers and score the row if it matches the schema.

Download CSV template

Choose CSV file

Upload and predict

This upload flow expects extracted feature columns, not raw packet capture files.

RESULT

Inference Result

The service still enforces the exact saved feature schema from the training artifacts.

Normal browsing: BENIGN | attack_probability=0.000001 | threshold=0.475000

Close Advanced Mode50 raw fields

Advanced Mode

Use this only if you already have the extracted network-flow metrics. The model expects all 50 values.

Load benign example

Load attack example

Reset fields

Timing

Flow Duration

Total lifetime of the connection or flow.

Enter numeric value

Fwd IAT Max

Max gap between forward packets in the flow.

Enter numeric value

36

16.3 DDoS Attack Sample

Spaces

abdullahyousaf13/_NimbusWatch_

Like 0

Running

Logs

App

Files

Community

Settings

GUIDED DEMO

NimbusWatch

NimbusWatch classifies one network flow at a time. It works on extracted numeric flow metrics, so a layman should start with guided examples or a prepared CSV template instead of typing raw network features manually.

Model
Isolation Forest

Threshold
0.475000

Feature Count
50

Input Type
Network Flow Metrics

PLAIN-ENGLISH GUIDE

Where do these values come from?

Network flow
A flow is a summarized conversation between endpoints, such as one client session talking to a server.

What the model reads
The model reads pre-extracted flow metrics like timing, packet sizes, rates, and TCP flags.

What a public user should do
Use the guided examples below, or upload one CSV row that already contains the trained feature columns.

GUIDED DEMO

Pick a prepared traffic example

Choose a scenario, then run prediction. Each example comes from the curated CICIDS-style data and already contains the required model fields.

BENIGN

Normal browsing
A typical non-malicious flow with steady packet behavior and normal acknowledgement traffic.
Dataset label: BENIGN
► Technical details

ATTACK

DDoS-like traffic
A high-volume flow pattern that resembles denial-of-service behavior with aggressive packet rates.
Dataset label: DDoS
► Technical details

ATTACK

Brute-force login attempt
Repeated login attempts can create bursty traffic and unusual session timing compared with normal use.
Dataset label: FTP-Patator
► Technical details

ATTACK

Web attack sample
This flow resembles a web application attack pattern and is included as a more targeted malicious example.
Dataset label: Web Attack ♦ Brute Force
► Technical details

Run selected example

Try your own template
Upload a UTF-8 CSV with exactly one data row and the exact trained columns. NimbusWatch will validate the headers and score the row if it matches the schema.

Download CSV template Choose CSV file Upload and predict

This upload flow expects extracted feature columns, not raw packet capture files.

RESULT

Inference Result

The service still enforces the exact saved feature schema from the training artifacts.

DDoS-like traffic: ATTACK | attack_probability=0.999991 | threshold=0.475000

Close Advanced Mode 50 raw fields

Advanced Mode
Use this only if you already have the extracted network-flow metrics. The model expects all 50 values.

Load benign example Load attack example

Reset fields

Timing

Flow Duration
Total lifetime of the connection or flow.
Enter numeric value

Fwd IAT Max
Max gap between forward packets in the flow.
Enter numeric value

37

16.4 Brute Force Attack Sample

Spaces

abdullahyousaf13/_NimbusWatch_

Like 0

Running

Logs

App

Files

Community

Settings

GUIDED DEMO

NimbusWatch

NimbusWatch classifies one network flow at a time. It works on extracted numeric flow metrics, so a layman should start with guided examples or a prepared CSV template instead of typing raw network features manually.

Model
Isolation Forest

Threshold
0.475000

Feature Count
50

Input Type
Network Flow Metrics

PLAIN-ENGLISH GUIDE

Where do these values come from?

Network flow
A flow is a summarized conversation between endpoints, such as one client session talking to a server.

What the model reads
The model reads pre-extracted flow metrics like timing, packet sizes, rates, and TCP flags.

What a public user should do
Use the guided examples below, or upload one CSV row that already contains the trained feature columns.

GUIDED DEMO

Pick a prepared traffic example

Choose a scenario, then run prediction. Each example comes from the curated CICIDS-style data and already contains the required model fields.

BENIGN

Normal browsing
A typical non-malicious flow with steady packet behavior and normal acknowledgement traffic.
Dataset label: BENIGN
▶ Technical details

ATTACK

DDoS-like traffic
A high-volume flow pattern that resembles denial-of-service behavior with aggressive packet rates.
Dataset label: DDoS
▶ Technical details

ATTACK

Brute-force login attempt
Repeated login attempts can create bursty traffic and unusual session timing compared with normal use.
Dataset label: FTP-Patator
▶ Technical details

ATTACK

Web attack sample
This flow resembles a web application attack pattern and is included as a more targeted malicious example.
Dataset label: Web Attack ♦ Brute Force
▶ Technical details

Run selected example

Try your own template
Upload a UTF-8 CSV with exactly one data row and the exact trained columns. NimbusWatch will validate the headers and score the row if it matches the schema.

Download CSV template Choose CSV file Upload and predict

This upload flow expects extracted feature columns, not raw packet capture files.

RESULT

Inference Result

The service still enforces the exact saved feature schema from the training artifacts.

Brute-force login attempt: ATTACK | attack_probability=0.981037 | threshold=0.475000

Close Advanced Mode 50 raw fields

Advanced Mode
Use this only if you already have the extracted network-flow metrics. The model expects all 50 values.

Load benign example Load attack example

Reset fields

Timing

Flow Duration
Total lifetime of the connection or flow.
Enter numeric value

Fwd IAT Max
Max gap between forward packets in the flow.
Enter numeric value

38

16.5 Web Attack Sample

Spaces

abdullahyousaf13/_NimbusWatch_

Like 0

Running

Logs

App

Files

Community

Settings

GUIDED DEMO

NimbusWatch

NimbusWatch classifies one network flow at a time. It works on extracted numeric flow metrics, so a layman should start with guided examples or a prepared CSV template instead of typing raw network features manually.

Model
Isolation Forest

Threshold
0.475000

Feature Count
50

Input Type
Network Flow Metrics

PLAIN-ENGLISH GUIDE

Where do these values come from?

Network flow

A flow is a summarized conversation between endpoints, such as one client session talking to a server.

What the model reads

The model reads pre-extracted flow metrics like timing, packet sizes, rates, and TCP flags.

What a public user should do

Use the guided examples below, or upload one CSV row that already contains the trained feature columns.

GUIDED DEMO

Pick a prepared traffic example

Choose a scenario, then run prediction. Each example comes from the curated CICIDS-style data and already contains the required model fields.

BENIGN

Normal browsing

A typical non-malicious flow with steady packet behavior and normal acknowledgement traffic.

Dataset label: BENIGN

► Technical details

ATTACK

DDoS-like traffic

A high-volume flow pattern that resembles denial-of-service behavior with aggressive packet rates.

Dataset label: DDoS

► Technical details

ATTACK

Brute-force login attempt

Repeated login attempts can create bursty traffic and unusual session timing compared with normal use.

Dataset label: FTP-Patator

► Technical details

ATTACK

Web attack sample

This flow resembles a web application attack pattern and is included as a more targeted malicious example.

Dataset label: Web Attack ♦ Brute Force

► Technical details

Run selected example

Try your own template

Upload a UTF-8 CSV with exactly one data row and the exact trained columns. NimbusWatch will validate the headers and score the row if it matches the schema.

Download CSV template

Choose CSV file

Upload and predict

This upload flow expects extracted feature columns, not raw packet capture files.

RESULT

Inference Result

The service still enforces the exact saved feature schema from the training artifacts.

Web attack sample: ATTACK | attack_probability=0.500124 | threshold=0.475000

Close Advanced Mode

50 raw fields

Advanced Mode

Use this only if you already have the extracted network-flow metrics. The model expects all 50 values.

Load benign example

Load attack example

Reset fields

Timing

Flow Duration

Total lifetime of the connection or flow.

Enter numeric value

Fwd IAT Max

Max gap between forward packets in the flow.

Enter numeric value

39

17. Conclusion

NimbusWatch successfully demonstrates a complete and academically strong hybrid-cloud malicious traffic detection system. It integrates data curation, preprocessing, supervised classification, reusable artifact export, stateless inference, serverless deployment thinking, support services, scheduling readiness, and cross-cloud portability into a single project.

The final implementation satisfies the key report requirements requested for the course:

- designing phase, options, and tradeoffs
- economic structure
- data handling, optimization, scheduling, and virtual machines
- support services, elasticity, and scalability
- system integration across clouds
- technical phase
- implementation phase
- deployment phase
- optimized cloud architecture
- research component

Most importantly, NimbusWatch demonstrates that cloud computing is not only about placing code on a remote server. It is about designing a complete system that uses cloud concepts intentionally. In NimbusWatch, model artifacts are separated from serving, serverless design reduces idle cost, support services improve observability, and hybrid-cloud packaging improves portability. Combined with strong recorded prediction performance, these qualities make NimbusWatch a well-rounded and submission-ready cloud computing project.